DIFF_DRIVE dynamics: derivation and analysis

Atom project

April 26, 2026

Abstract

This document derives the planar dynamics of a two-wheeled differential-drive vehicle, starting from the rolling-without-slipping wheel constraints and ending at the analytic Jacobians used by the linearisation. It is the canonical reference for the C++ implementation in

bl bla blaaa ssss `dynamics/diff_drive/core/` and the numerical companion notebook in `analysis/dynamics_tour.ipynb`.

Contents

1 Setup and notation **2**
- 1.1 Frames . . . 2
- 1.2 Configuration and velocities . . . 2
- 1.3 Geometric and actuator parameters . . . 2
- 1.4 Wheel inputs . . . 2

2 Wheel-contact constraints **3**
- 2.1 Wheel positions and contact velocities . . . 3
- 2.2 Lateral constraint (no side slip) . . . 3
- 2.3 Longitudinal constraint (pure rolling) . . . 3
- 2.4 Pfaffian form . . . 4
- 2.5 The constraint is non-holonomic . . . 4
- 2.6 Consistency with the implementation . . . 4

3 Kinematics: wheel speeds and body twist **4**
- 3.1 Forward and inverse maps . . . 5
- 3.2 Instantaneous centre of rotation . . . 5
- 3.3 Special operating modes . . . 5
- 3.4 The kinematic unicycle interpretation . . . 5

4 Dynamics: motor lag and the state ODE **6**
- 4.1 Commanded body twist . . . 6
- 4.2 Motor lag . . . 6
- 4.3 Full state ODE . . . 6
- 4.4 Why 5-D, not 7-D? . . . 7
- 4.5 Slip . . . 7

5 Linearisation about an operating point **7**
5.1 State Jacobian A . . . 7
5.2 Control Jacobian B . . . 8
5.3 Linear rollout error . . . 8
5.4 Verification routes . . . 8

6 Verification **9**
6.1 Test crosswalk . . . 9
6.2 Tests still to add . . . 9
6.3 Editing discipline . . . 10

1 Setup and notation

1.1 Frames

We work entirely in the plane. The world frame is inertial with axes $(\hat{x}_w, \hat{y}_w)$. The body frame is rigidly attached to the vehicle at the geometric centre of the wheel axle, with $\hat{x}_b$ pointing forward (the direction of travel when both wheels rotate equally) and $\hat{y}_b$ pointing to the left:

$$\hat{x}_b = \begin{pmatrix} \cos\theta \\ \sin\theta \end{pmatrix}, \qquad \hat{y}_b = \begin{pmatrix} -\sin\theta \\ \cos\theta \end{pmatrix}. \tag{1}$$

1.2 Configuration and velocities

The configuration is

$$q \;=\; (p_x,\, p_y,\, \theta) \;\in\; \mathbb{R}^2 \times S^1, \tag{2}$$

where (p_x, p_y) is the world-frame position of the wheel-axle midpoint and θ is the heading. We use the body twist (v, ω) as the dynamical coordinates for translational and angular velocity:

$$v \;=\; \dot{p}_x \cos\theta + \dot{p}_y \sin\theta, \tag{3}$$

$$\omega \;=\; \dot{\theta}. \tag{4}$$

Once the no-side-slip constraint of §2 is imposed, this reduces to $(\dot{p}_x, \dot{p}_y) = v\,\hat{x}_b$.

1.3 Geometric and actuator parameters

Track width W — distance between the left and right wheel contacts. Default 0.2 m.

Wheel radius r_w — used only when converting between commanded wheel angular velocity $\dot{\phi}$ and rim velocity $v_w = r_w\,\dot{\phi}$.

Motor time constant τ — first-order lag from commanded to realised body velocities. Default 0.05 s.

1.4 Wheel inputs

The control input is the pair of commanded wheel *rim* velocities $u = (v_L,\, v_R)$, with $v_L = r_w\,\dot{\phi}_L$ and similarly for the right wheel. We use v_L, v_R rather than $\dot{\phi}_L$, $\dot{\phi}_R$ throughout because the wheel radius is absorbed at the controller boundary and never appears in the body dynamics. The `Control<Scalar>` type in `types.hpp` stores exactly this pair.

2 Wheel-contact constraints

The vehicle has two wheels, each treated as an ideal rigid disk rolling without slipping on a planar ground. Each contact point yields two scalar constraints — one along the rolling direction and one perpendicular to it. Below we derive both, show that the lateral pair degenerates into a single body-level constraint, and prove that this constraint is non-holonomic.

2.1 Wheel positions and contact velocities

The left wheel sits a distance $W/2$ along $\hat{y}_b$ from the body origin; the right wheel sits the same distance along $-\hat{y}_b$:

$$r_L = \begin{pmatrix} p_x - \frac{1}{2}W\sin\theta \\ p_y + \frac{1}{2}W\cos\theta \end{pmatrix}, \qquad r_R = \begin{pmatrix} p_x + \frac{1}{2}W\sin\theta \\ p_y - \frac{1}{2}W\cos\theta \end{pmatrix}. \tag{5}$$

Differentiating in time,

$$\dot{r}_L = \begin{pmatrix} \dot{p}_x - \frac{1}{2}W\,\dot{\theta}\cos\theta \\ \dot{p}_y - \frac{1}{2}W\,\dot{\theta}\sin\theta \end{pmatrix}, \qquad \dot{r}_R = \begin{pmatrix} \dot{p}_x + \frac{1}{2}W\,\dot{\theta}\cos\theta \\ \dot{p}_y + \frac{1}{2}W\,\dot{\theta}\sin\theta \end{pmatrix}. \tag{6}$$

2.2 Lateral constraint (no side slip)

Each wheel cannot move along its axle. Both axles are aligned with $\hat{y}_b$, so for each wheel the constraint is $\dot{r}_w \cdot \hat{y}_b = 0$:

$$\begin{aligned} \dot{r}_L \cdot \hat{y}_b &= -\dot{p}_x\sin\theta + \tfrac{1}{2}W\,\dot{\theta}\cos\theta\sin\theta + \dot{p}_y\cos\theta - \tfrac{1}{2}W\,\dot{\theta}\sin\theta\cos\theta \\ &= -\dot{p}_x\sin\theta + \dot{p}_y\cos\theta. \end{aligned} \tag{7}$$

The $\dot{\theta}$-dependent cross terms cancel because the wheel offset is itself along $\hat{y}_b$, so the spin-induced contact velocity is along $\hat{x}_b$, not $\hat{y}_b$. The right wheel gives the same expression. Both lateral constraints therefore collapse into a single body-level Pfaffian constraint:

$$\boxed{-\sin\theta\,\dot{p}_x + \cos\theta\,\dot{p}_y = 0.} \tag{8}$$

2.3 Longitudinal constraint (pure rolling)

The forward velocity of each wheel contact equals its rim velocity:

$$\dot{r}_L \cdot \hat{x}_b = v_L, \qquad \dot{r}_R \cdot \hat{x}_b = v_R. \tag{9}$$

Expanding,

$$v_L = \dot{p}_x\cos\theta + \dot{p}_y\sin\theta - \tfrac{1}{2}W\,\dot{\theta} = v - \tfrac{1}{2}W\,\omega, \tag{10}$$

$$v_R = \dot{p}_x\cos\theta + \dot{p}_y\sin\theta + \tfrac{1}{2}W\,\dot{\theta} = v + \tfrac{1}{2}W\,\omega. \tag{11}$$

Solving for the body twist,

$$\boxed{v = \tfrac{1}{2}(v_L + v_R), \qquad \omega = \frac{v_R - v_L}{W}.} \tag{12}$$

This is the map used at the top of `DiffDriveDynamics::continuous` in `core/dynamics.hpp`.

2.4 Pfaffian form

Stacking the three constraints,

$$\underbrace{\begin{bmatrix} -\sin\theta & \cos\theta & 0 \\ \cos\theta & \sin\theta & -\frac{1}{2}W \\ \cos\theta & \sin\theta & +\frac{1}{2}W \end{bmatrix}}_{A(q)} \begin{bmatrix} \dot{p}_x \\ \dot{p}_y \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} 0 \\ v_L \\ v_R \end{bmatrix}. \tag{13}$$

Row 1 is homogeneous in $\dot{q}$ — no actuator drives it. This is the non-holonomic part. Rows 2–3 are inhomogeneous, driven by the wheel commands.

2.5 The constraint is non-holonomic

A Pfaffian constraint $a(q) \cdot \dot{q} = 0$ is integrable to a function $h(q) = \text{const}$ if and only if the dual distribution $\Delta = \ker a(q)$ is involutive (Frobenius theorem). For (8),

$$\Delta = \operatorname{span}\{X_1, X_2\}, \qquad X_1 = \begin{pmatrix} \cos\theta \\ \sin\theta \\ 0 \end{pmatrix}, \quad X_2 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}, \tag{14}$$

where X_1 generates pure forward motion and X_2 generates pure rotation in place. The Lie bracket is

$$[X_1, X_2] \;=\; \frac{\partial X_1}{\partial q} X_2 \;-\; \frac{\partial X_2}{\partial q} X_1 \;=\; \begin{pmatrix} -\sin\theta \\ \cos\theta \\ 0 \end{pmatrix} \;-\; 0. \tag{15}$$

This vector is precisely the body-lateral direction (8) forbids, so $[X_1, X_2] \notin \Delta$. The distribution is therefore non-involutive, (8) is non-integrable, and the system is non-holonomic. □

Remark (Chow's theorem and parking). The set $\{X_1,\, X_2,\, [X_1, X_2]\}$ spans $\mathbb{R}^3$ at every q. By Chow's theorem the system is locally controllable: any configuration (p_x, p_y, θ) can be reached by combining forward motion and rotation, even though the vehicle has only two control inputs and cannot translate sideways instantaneously. Parallel parking is the everyday demonstration of this fact.

2.6 Consistency with the implementation

The state ODE in `dynamics.hpp` is written directly in body coordinates,

$$\dot{p}_x = v\cos\theta, \qquad \dot{p}_y = v\sin\theta, \tag{16}$$

which substituted into (8) gives $-v\sin\theta\cos\theta + v\sin\theta\cos\theta = 0$ identically. The constraint is built into the parameterisation; the simulator does not need projection or Lagrange multipliers to enforce it at runtime. The numerical verification of this identity over a long rollout will live in `analysis/dynamics_tour.ipynb` (a future "Test 5: constraint satisfaction" cell).

3 Kinematics: wheel speeds and body twist

The constraints in §2 relate wheel rim velocities (v_L, v_R) to body twist (v, ω) via (12). This section unpacks the consequences of that map: how to convert between actuator space and body space, the geometric picture, and the special operating modes that come up later.

3.1 Forward and inverse maps

The forward map (wheel commands → body twist), restated from (12),

$$v = \tfrac{1}{2}(v_L + v_R), \qquad \omega = \frac{v_R - v_L}{W}. \tag{17}$$

The inverse map (desired body twist → wheel commands) follows immediately from the rolling constraints in §2:

$$v_L = v - \tfrac{1}{2}W\,\omega, \qquad v_R = v + \tfrac{1}{2}W\,\omega. \tag{18}$$

Both maps are linear and invertible, so the kinematic state of the differential drive is captured equally well in either parameterisation. Internally the dynamics core in §4 carries (v, ω) as state and treats (v_L, v_R) as the boundary input, because the body twist is what couples to the motor lag and to Box2D.

3.2 Instantaneous centre of rotation

For any non-zero ω the body's instantaneous motion is a rotation about a point fixed in the world frame at signed distance R along $\hat{y}_b$, where

$$R \;=\; \frac{v}{\omega} \;=\; \frac{W}{2} \cdot \frac{v_L + v_R}{v_R - v_L}. \tag{19}$$

This is the instantaneous centre of rotation (ICR). R is the signed turn radius: positive when turning left (CCW), negative when turning right.

3.3 Special operating modes

Pure forward. $v_L = v_R \implies \omega = 0,\ v = v_L = v_R$. Straight-line motion; ICR at infinity.

Pure rotation. $v_L = -v_R \implies v = 0,\ \omega = 2v_R/W$. The vehicle spins about the body origin; ICR coincident with the axle midpoint.

One wheel locked. $v_L = 0 \implies v = v_R/2,\ \omega = v_R/W,\ R = W/2$. The vehicle pivots about the locked wheel; ICR at the locked wheel's contact point.

These three modes form a useful test set — each isolates one component of (17) from the others, and Test 2 and Test 3 of `dynamics_tour.ipynb` exercise the first two directly (§6).

3.4 The kinematic unicycle interpretation

Once the no-slip constraint (8) is imposed, the planar motion of the vehicle is entirely described by a unicycle on (p_x, p_y, θ) driven by the body twist (v, ω):

$$\dot{p}_x = v\cos\theta, \tag{20}$$

$$\dot{p}_y = v\sin\theta, \tag{21}$$

$$\dot{\theta} = \omega. \tag{22}$$

The differential-drive structure shows up only in (i) how (v, ω) are produced by the wheels — equation (17) above — and (ii) the motor lag in §4 that separates commanded from realised body twist. Many planning algorithms operate at the unicycle level and only invert through (18) at the lowest control layer; we keep both layers explicit here because the motor lag couples them at the dynamics level.

4 Dynamics: motor lag and the state ODE

The kinematic map of §3 treats wheel commands as instantaneously producing the desired body twist. Real motors have inertia, controller dynamics, and current limits; their realised speed lags the command. We fold all of this into a single first-order lag on the body velocities.

4.1 Commanded body twist

The wheel command pair $u = (v_L, v_R)$ produces a commanded body twist via the forward kinematic map (17):

$$v_{\text{cmd}} = \tfrac{1}{2}(v_L + v_R), \qquad \omega_{\text{cmd}} = \frac{v_R - v_L}{W}. \tag{23}$$

"Commanded" to distinguish from the realised body velocities (v, ω) that appear in the state.

4.2 Motor lag

The realised body velocities track the commanded values through a first-order lag with time constant τ:

$$\dot{v} \;=\; \frac{1}{\tau}\big(v_{\text{cmd}} - v\big), \tag{24}$$

$$\dot{\omega} \;=\; \frac{1}{\tau}\big(\omega_{\text{cmd}} - \omega\big). \tag{25}$$

The same time constant is used for both channels. We could carry separate τ^v and τ^ω if a motor identification campaign warranted it, but in practice the difference is absorbed into the per-episode domain randomisation in `training/`.

Remark (Why first-order, not second-order?). A second-order lag would track an additional state per channel (acceleration), capturing the motor's torque dynamics more faithfully. For the wheel masses and drive sizing we expect, the second-order modes are well above the policy's control bandwidth and a first-order fit is adequate. The model can be promoted to second-order later without disturbing anything outside this section.

4.3 Full state ODE

Stack the position, heading, and body twist into a single state vector $\mathbf{x} \in \mathbb{R}^5$:

$$\mathbf{x} \;=\; \begin{pmatrix} p_x & p_y & \theta & v & \omega \end{pmatrix}^\top. \tag{26}$$

Combining the unicycle kinematics (22) with the motor lag (24)–(25) and substituting (23):

$$\dot{\mathbf{x}} \;=\; f(\mathbf{x}, u) \;=\; \begin{pmatrix} v\cos\theta \\ v\sin\theta \\ \omega \\ \dfrac{1}{\tau}\big(\tfrac{1}{2}(v_L + v_R) \;-\; v\big) \\ \dfrac{1}{\tau}\left(\dfrac{v_R - v_L}{W} \;-\; \omega\right) \end{pmatrix}. \tag{27}$$

This is the equation `DiffDriveDynamics::continuous` in `core/dynamics.hpp` implements verbatim. The integrator (`rk4_step` in `core/integrators.hpp`) treats u as zero-order-hold over a step of length Δt and advances $\mathbf{x}$ accordingly.

4.4 Why 5-D, not 7-D?

There are three reasonable parameterisations:

3-D kinematic. State (p_x, p_y, θ), input (v, ω). No actuator dynamics — fine for kinematic planners but useless for tracking control.

5-D body-lag. State $(p_x, p_y, \theta, v, \omega)$, input (v_L, v_R). Single first-order lag at the body level. *This document.*

7-D wheel-lag. State $(p_x, p_y, \theta, \dot{\phi}_L, \dot{\phi}_R, \ldots)$, input (v_L, v_R). Per-wheel lag, body twist computed downstream.

We chose the middle option: it captures the dominant transient (motor lag) without paying for two redundant per-wheel states, and it keeps the embedded MPC problem small enough that linearised solvers like TinyMPC stay in their sweet spot. If a future task demonstrates per-wheel lag matters — e.g., uneven loading or one motor saturating before the other — the model expands to 7-D, but evidence-driven, not preemptively.

4.5 Slip

The model assumes pure rolling. The kinematic map (17) carries no slip term, and the motor lag operates on the resulting body velocities directly. Domain randomisation on W, τ, and the per-wheel gain absorbs much of what slip would contribute in practice. If a deployed policy starts exploiting non-physical kinematic behaviour — e.g., turning faster than ω_{cmd} permits — a velocity-dependent slip term such as

$$\omega_{\text{achieved}} \;=\; \omega_{\text{cmd}}\,(1 - \alpha\,|v|) \tag{28}$$

can be folded into (23), but only as a corrective response to observed exploit, not as an a-priori complication.

5 Linearisation about an operating point

For controller design (LQR, MPC) and stability analysis we need the Jacobians $A = \partial f/\partial \mathbf{x}$ and $B = \partial f/\partial u$ evaluated at an operating point $(\mathbf{x}^\star, u^\star)$. They are sparse and analytic, with no allocation and only a handful of elementary functions — exactly what an embedded linear MPC solver wants.

5.1 State Jacobian A

Differentiating (27) entry-by-entry with respect to $\mathbf{x}$,

$$A(\mathbf{x}) \;=\; \frac{\partial f}{\partial \mathbf{x}} \;=\; \begin{bmatrix} 0 & 0 & -v\sin\theta & \cos\theta & 0 \\ 0 & 0 & v\cos\theta & \sin\theta & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & -1/\tau & 0 \\ 0 & 0 & 0 & 0 & -1/\tau \end{bmatrix}. \tag{29}$$

Structure worth noticing:

Position rows. Rows 1–2 are exactly the kinematic-unicycle rows: position rates depend on (v, θ) but on neither position nor ω.

Heading row. Row 3 is trivial — $\dot{\theta}$ depends only on ω.

Velocity rows. Rows 4–5 are pure first-order decay, independent of *anything else* in the state. The motor lag does not couple to position or heading.

Operating-point dependence. Only column 3 (the θ-derivatives) and column 4 (the v-derivatives in rows 1–2) carry trigonometric terms. Everything else is a constant.

5.2 Control Jacobian B

The control affects only the velocity rows of f, through the commanded twist (23). Differentiating with respect to $u = (v_L, v_R)$:

$$B \;=\; \frac{\partial f}{\partial u} \;=\; \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ \dfrac{1}{2\,\tau} & \dfrac{1}{2\,\tau} \\ -\dfrac{1}{W\,\tau} & \dfrac{1}{W\,\tau} \end{bmatrix}. \tag{30}$$

B is constant: it depends on neither $\mathbf{x}$ nor u. The signs of the bottom row encode left-handed-versus-right-handed yaw: a positive v_R alone produces positive ω (CCW yaw); a positive v_L alone produces negative ω (CW). Row 4 is symmetric in the two wheels because the body forward velocity is their average.

5.3 Linear rollout error

Near the operating point the nonlinear flow agrees with the linear approximation up to second order. By Taylor's theorem, for fixed $u^\star$,

$$f(\mathbf{x}^\star + \delta\mathbf{x},\, u^\star) \;-\; f(\mathbf{x}^\star,\, u^\star) \;-\; A(\mathbf{x}^\star)\,\delta\mathbf{x} \;=\; \mathcal{O}\big(\|\delta\mathbf{x}\|^2\big). \tag{31}$$

Plotting the residual norm against $\|\delta\mathbf{x}\|$ on log–log axes therefore gives a slope of exactly 2 when the analytic A in (29) is correct. This is the strongest single test of the linearisation, and the one Test 4 of `dynamics_tour.ipynb` runs (see §6).

5.4 Verification routes

We treat (29)–(30) as the definitive analytic form, then verify them by three independent means:

1. **Finite differences in C++.** Compute centred-difference estimates of A and B at random operating points and compare entry-wise. Cheap; catches sign flips, factor-of-two errors, and missing entries. *To be added as a doctest case in `core/tests/`.*

2. **Forward-mode autodiff in C++.** Instantiate `DiffDriveDynamics<Eigen::AutoDiffScalar<Eigen::Ma` and read A off the autodiff result directly; compare to the analytic form. *To be added once an autodiff dependency is wired in.*

3. **Symbolic re-derivation in Julia.** `analysis/jacobian_verify.jl` (planned) re-derives (29)–(30) symbolically with `Symbolics.jl` and compares numerically at random operating points. This catches errors in the LaTeX itself, not just in the C++.

The slope-2 numerical test in the notebook (Test 4) sits across all three routes: it verifies the C++ implementation *and* the analytic form simultaneously, without referring to any third source. As long as that test passes, A is correct in the dynamic — "does the simulation flow agree with the linear flow near the operating point?" — sense, even if some hand-derived entries happened to be wrong they would have to be wrong in compensating ways. It is the single test most worth running on every CI build.

6 Verification

The C++ in `core/` and the equations in this document must stay in agreement. We pin them together with two layers of tests: doctest unit tests in `core/tests/` that run on every CI build, and the Jupyter notebook `analysis/dynamics_tour.ipynb` that exercises the same code from Python with plots and looser numerical tolerances. This section maps each result derived above to the test that verifies it.

6.1 Test crosswalk

Notebook test	Verifies	Equation
1. Motor-lag step response	First-order lag on $\dot{v}$	(24)
2. Pure rotation	No side slip ($v = 0$, $p_x = p_y = 0$)	(8)
	Yaw rate from differential commands	(17)
3. Constant arc	Steady-state turn radius $R = v_{ss}/\omega_{ss}$	(19)
4. Linearisation residual	Analytic A matches the nonlinear flow to first order	(29), (31)

The `core/tests/` doctest binary runs three lower-level checks: state derivative at the origin under symmetric control, single-step RK4 sanity, and finiteness of A and B. It is the build-time canary; the notebook is the analyst-facing companion that produces plots.

6.2 Tests still to add

Two checks are derived in this document but not yet wired into either test layer:

1. **Constraint satisfaction along long rollouts.** (8) is identically satisfied by the parameterisation (27), so this is essentially a regression test against the position equations being edited. Future Test 5 of the notebook: roll out for several seconds under a varying control sequence and assert
$$\left| -\sin\theta(t)\,\dot{p}_x(t) + \cos\theta(t)\,\dot{p}_y(t) \right| < 10^{-12}$$
at every time step.

2. **Finite-difference Jacobian comparison in C++.** §5 derives A and B analytically; the doctest binary should compare them to centred-difference estimates at randomly sampled operating points. This is the cheapest of the three verification routes from §5 and the highest-value to add first.

When either of these is added, this section's table grows by one row and the corresponding bullet here is removed.

6.3 Editing discipline

The chain of authority across the project is:

$$\text{LaTeX } (\textit{here}) \longrightarrow \text{C++ } (\texttt{core/}) \longrightarrow \text{Python } (\texttt{bindings/}, \text{ notebook}).$$

If the LaTeX changes, the C++ must change to match before the doctest binary is re-run. If the C++ changes without the LaTeX, the doctest will likely still pass — but the document will be wrong, and the next person to read it will derive the wrong control law from the wrong equation. The reviewer's job on any `core/` pull request is to confirm the LaTeX is updated in lockstep.